
PROJECT PROPOSAL

Generation 11 - Software Engineering - Group 01
Team11 - 3Bugs

GitHero - GitHero is a platform that helps fresh developers learn and apply real-world Git workflows through interactive practice and guided simulations.

Prepared By: Team3Bugs
IDTB110266 Yan Sovanpisoth
IDTB110380 Vun Devit
IDTB100245 Lang Mingyang

Prepared For:
Lect. Mr. Kimlong Ngin

Table of Content

I. Introduction.....	2
Background.....	2
Objectives.....	2
Scope.....	2
Impact.....	2
II. Methods and Tools.....	2
Agile Methodology.....	2
CNN (Future Enhancement).....	2
MVC Architecture.....	3
ERD.....	3
3NF.....	3
Dataset.....	3
Data Process.....	3
UML.....	4
III. Plan.....	4
Phase 1: Requirement Analysis (Week2-4).....	4
Phase 2: Design (Week5).....	4
Phase 3: Development (Week6-10).....	5
Phase 4: Deployment.....	5
IV. Expected Result.....	5
V. Demo Prototype.....	5
VI. Conclusion.....	5

I. Introduction

Background

Git and GitHub are essential tools in modern software development, enabling version control and collaboration. However, beginners often struggle due to the complexity of commands and lack of interactive learning environments. GitHero is designed to solve this issue by providing a structured, interactive platform that combines learning, practice, and simulation of real Git workflows.

Objectives

- Develop a full-stack web application for Git learning
- Simulate real Git workflows (commit, repository, branch concepts)
- Provide structured lessons, quizzes, and progress tracking

Scope

The platform focuses on the following scope:

- Target Users: First-year to second-year university students new to Git and GitHub.
- Content Coverage: Core Git workflow commands and GitHub collaboration features (branches, pull requests, issues).
- Platform: Web-based (accessible from any browser, no installation required).
- Features: Lesson modules, interactive command simulator, GitHub UI visualization panel, progress dashboard, Community Hub, Quizzes

Impact

- Simplifies Git learning for beginners
- Provides hands-on practical experience
- Improves student understanding of version control
- Bridges the gap between theory and real-world development

II. Methods and Tools

Agile Methodology

- Sprint-based development
- Iterative feature delivery
- Continuous improvement

CNN (Future Enhancement)

- Learning behavior prediction

- Personalized recommendations
- Performance analytics

MVC Architecture

MVC (Model-View-Controller) is a software architecture pattern that separates an application into **three parts** so that the code is easier to develop, maintain, and modify.

The platform follows the Model-View-Controller (MVC) architectural pattern:

Layer	Responsibilities	Technologies
Model	Data management: lessons, user progress, command history, user accounts	MYSQL MongoDB Supabase
View	User interface: split-screen IDE panel, GitHub UI panel, dashboard, buttons, menus	HTML, CSS, JavaScript, ReactJS, TailwindCSS.
Controller	Business logic: command validation, progress tracking, session management	Node.js Express.js Python

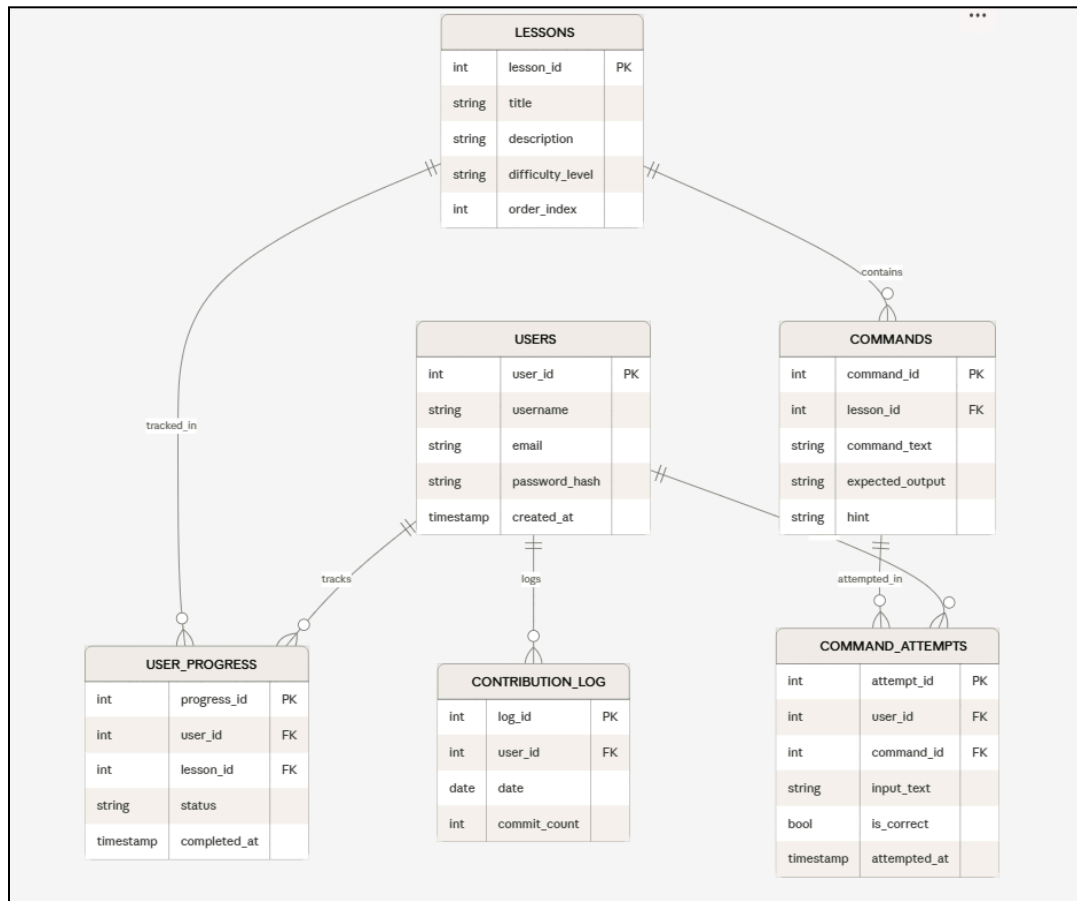
Example:

A user clicks "Run Git Command."

The Controller:

1. Receives the command from the View.
2. Checks whether the command is valid.
3. Requests the Model to save the command history.
4. Returns the result.
5. The View displays the result to the user.

ERD



3NF (Must already be in 2NF)

Since our database is in **3NF**, it has already met the requirements of **1NF** and **2NF**. Data is stored in **separate tables to avoid duplication**, and **every attribute depends only on its table's primary key**, which is easier to manage and reduces errors.

Key entities include:

- **Users** (
 User_id,
 Username,
 Email,
 Password_hash,
 Created_at)
- **Lessons** (
 lesson_id,
 title,
 description,
 difficulty_level,
 Order_index)

- **Commands** (command_id, lesson_id, command_text, expected_output, hint)
- **UserProgress** (progress_id, user_id, lesson_id, status, completed_at)
- **CommandAttempts** (attempt_id, user_id, command_id, input_text, is_correct, attempted_at)
- **ContributionLog** (log_id, user_id, date, commit_count)

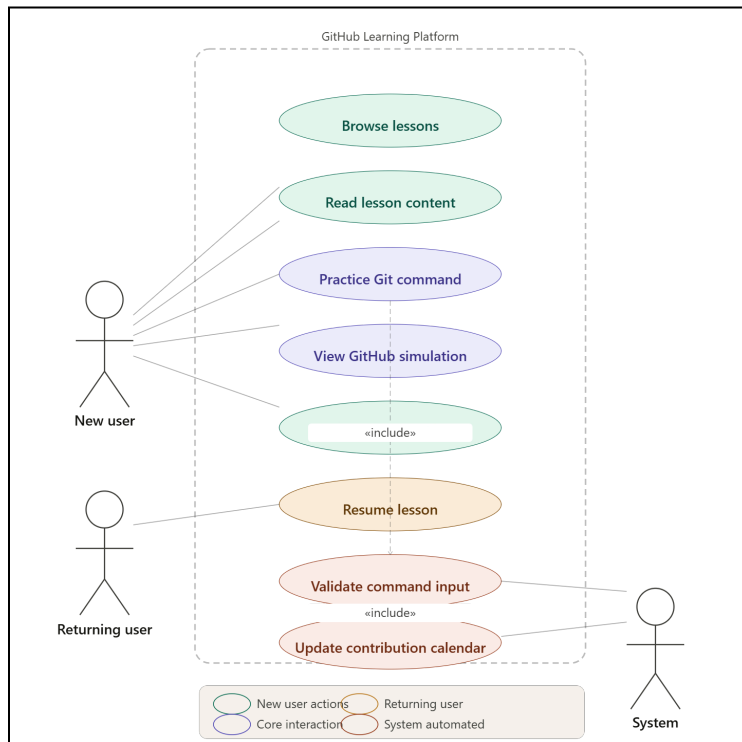
Dataset

- **User accounts** (User ID, Username, Email, Password (encrypted), Account creation date) and authentication data
- **Learning content** (courses titles , lessons descriptions , quizzes questions)
- **Git simulation data** (repositories, commits, progress tracking)

Data Process

- User authentication via API
- Lesson retrieval and quiz submission
- Progress tracking updates
- Git command simulation mapped to database operations

UML



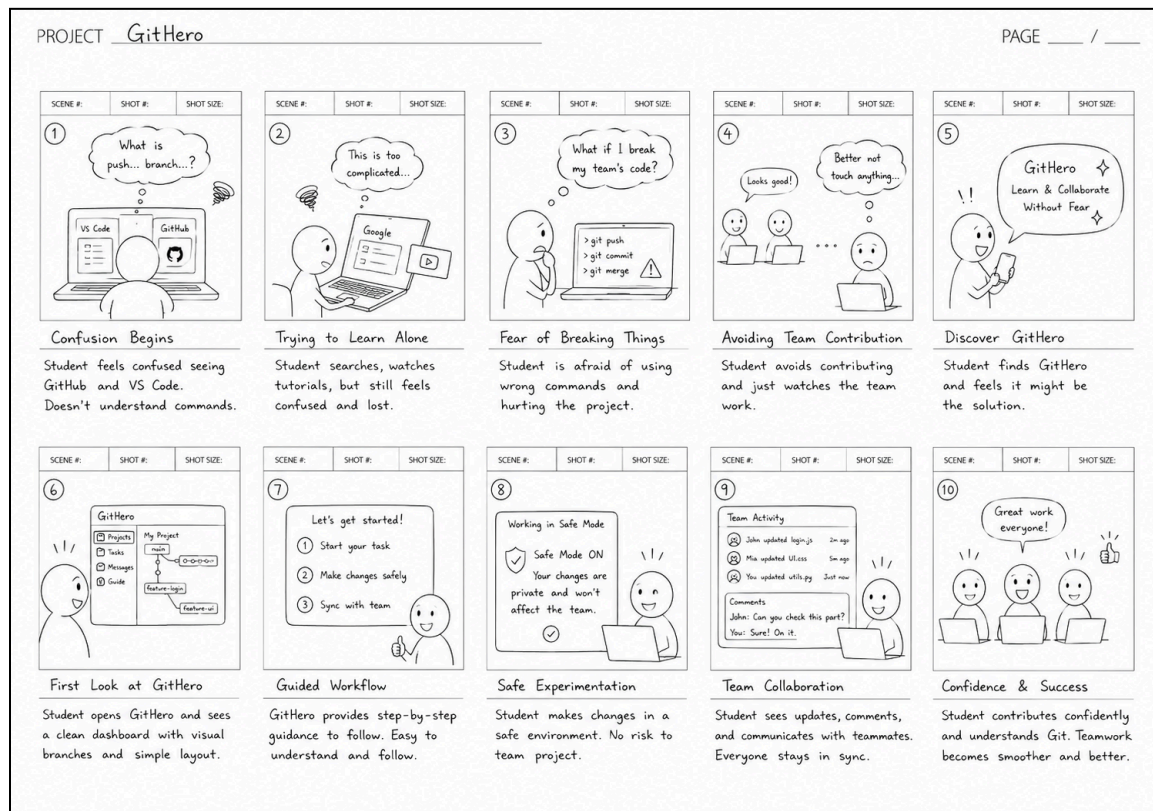
III. Plan

Phase 1: Requirement Analysis (Week2-4)

- Define system scope
- Identify user needs

Phase 2: Design (Week5)

- Design UI in Figma
- Storyboarding



Phase 3: Development (Week6-10)

Timeline and WBS (Work Breakdown Structure)

Week	Goal	Description	Responsible
Week6	User Authentication	Able user to signup/signin, linking from Landing page to Dashboard	Soth
Week7	Core Feature I	Implement lessons, sharing platform,	All member

	(lessons, platform)	quizzes and other feature related to educated users	
Week8	Core Feature II (IDE, Repository)	Core feature of GitHero, user able to practice writing commands without affect their codes, Repository visualization	Devit + Soth
Week9	Testing and Finalization	Testing, debug, optimize codes and finalize system structure	All member
Week10	Documentation	Document system architecture, slides	All member

Phase 4: Deployment

Deploy this project to the public and open it for testing

IV. Expected Result

GitHero is expected to improve beginners' understanding of Git and GitHub through interactive, simulation-based learning.

- Users will be able to practice real Git operations such as commit, branch, and merge in a safe environment, reducing the learning curve compared to traditional methods.
- The system will also increase engagement through structured lessons, quizzes, and progress tracking

V. Demo Prototype

<https://www.figma.com/design/uROfFAUsX9IWibMaJDtaTo/GitHero?node-id=0-1&t=KPJD6UBrPFHw4dOZ-1>

VI. Conclusion

GitHero is a structured learning platform designed to simplify Git and GitHub learning through simulation and guided education. The system integrates modern software engineering practices including MVC architecture, ERD design, UML modeling, and database normalization to ensure scalability and maintainability.